

PNN におけるスレッド数の変化に関する高速化について

Acceleration of Computation for Probabilistic Neural Network by Varying the Number of Threads

指導教員 保谷 哲也 教授

3802 奥山 響

1. まえがき

近年、画像認識や自然言語処理などの分野で大きな成功を収めている深層学習は、複雑な非線形性を表現する強力な学習手法として広く用いられている。しかしながら、その学習過程においては、重みやバイアスの初期値設定、最適化アルゴリズムの選択、ネットワーク構造の設計など、多くの課題が存在する。特に、誤差逆伝播法に基づく学習は、勾配消失や局所最適解への収束といった問題に直面することがある。そこで、本研究では、これらの課題を克服する可能性を秘めた確率的ニューラルネットワーク（Probabilistic Neural Network; PNN）に着目した。PNN は、単一の平滑化パラメータのみで高速な学習が可能であり、確率的な出力に基づいた分類を行うため、外れ値の影響を受けにくいという特徴を有している。しかしながら、従来の PNN では、学習データがそのままパターン層に格納されるため、データ数の増加に伴い分類処理時間が増大するという課題がある。本研究では、CPU と GPU の並列処理を活用することで、この処理時間の高速化を目指す。

2. 原理・方法

2.1 CPU と GPU の違い

CPU と GPU の大きな違いは、その並列性にある。CPU は一つまたは数個のコア（処理装置）を持ち、一つずつ命令を実行する。一方、GPU は数百から数千のコアを持ち、多くの命令を同時に実行することが可能である¹⁾。

2.2 CUDA

CUDA とは、NVIDIA 製の GPU が最適な性能を発揮できる環境を提供するために NVIDIA 自身が 2007 年に開発した並列演算処理を可能とするコンピューティングプラットフォームである。GPU の並列処理を高速化することが可能で、科学技術計算や機械学習、ビッグデータ解析、画像・動画処理などさまざまな分野で活用されている²⁾。

2.3 PNN

PNN とは、Specht によって提唱された多層ニューラルネットワークの一種であり、入力層、パターン層、結合層、出力層から構成される³⁾。図 1 に PNN の構造図を示す。

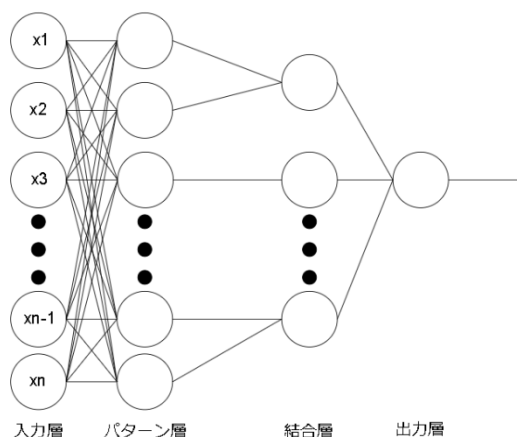


図 1 PNN の構造

パターン層は入力データと訓練データとの類似度を計算する層であり、各訓練データに対応するニューロンを持ち、放射基底関数（RBF）を用いて類似度を計算する³⁾。結合層は各クラスに対応するニューロンを持つ層であり、パターン層からの出力をクラスごとに集計する。出力層は各クラスの事後確率を出力する層であり、結合層からの出力を正規化して確率値として出力する³⁾。

2.4 スレッド数を変更し、処理時間を測定

スレッド数とは同時に処理できる命令の最大数を表す。8 コア / 16 スレッドである CPU の場合、同時に処理できる命令数は 8 であるがひとつのコアでふたつまで、最大で 16 の命令を同時に処理することが可能である。そのためコア数を物理コア数、スレッド数を論理コア数とも呼ぶ⁴⁾。

本実験では Python を用いて、CPU による並列実験を行った。PNN のプログラムでスレッド数を変更し、処理時間を測定した上で、比較を行った。また、GPU においても並列実験を行い、結果を比較した。

GPU 並列処理においてスレッドの集合であるブロックの x-y 次元を変化させ、処理時間の計測をした上で、比較を行った。また、本実験に使用する GPU でのブロックにおける最大値は 1024 であったため、x と y の積が 1024 となるように数値を設定した。

3. 結果

3.1 実行環境

実験では Python 言語を用い、CPU 上での並列環境において PNN によるパターン分類を行うプログラム、および C++言語と CUDA を用い GPU 上での並列環境におけるプログラムを使用した。

次に、実験に使用した PC の性能及び、実行環境等について表 1 に示す。

表 1 実行環境

CPU	AMD Ryzen 7 3700X 8-core processor
GPU	NVIDIA GeForce RTX 3060
開発・実行環境	Visual Studio2022
Python	Version3.13
CUDA	Version12.6

次に使用したデータセットの詳細について表 2 に示す。

表2 使用したデータセット

名前	学習データ数	テストデータ数	次元数	クラス数
balance-scale	438	187	4	3
ionosphere	200	151	33	2
tic-tac-toe	663	295	9	2
wdbc	398	171	30	2
yeast	890	594	8	10
cmc	1000	473	9	3
segmentation	210	2100	18	7
abalone	2088	2089	7	3
page-blocks	2000	3473	10	5
sat	4435	2000	36	6
pendigits	7494	3498	16	10
optdigits	3823	1797	64	10
letter-recognition	16000	4000	16	26
isolet	2252	1559	617	26
MNIST10k	10000	1000	784	10

3.2 実験結果

2.3の通り、スレッド数を変更し、それぞれのデータセットに対する分類処理時間の計測を行った。結果を図2に示す。

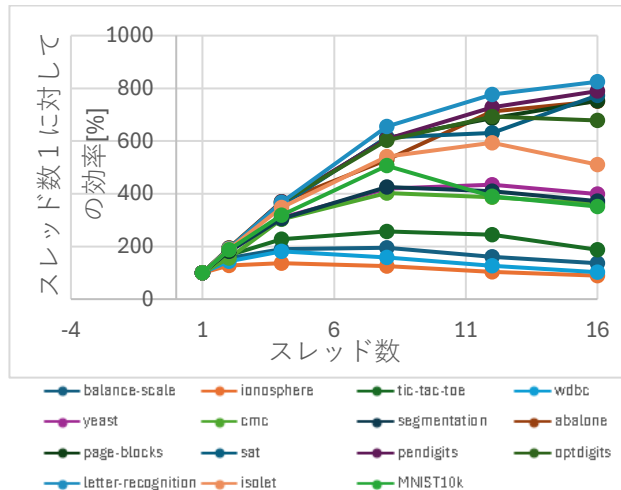


図2 CPUの並列実験

次にGPUにおいてもCPUの場合同様に並列実験を行った。その結果をCPUを用いた場合と比較したものを図3に示す。

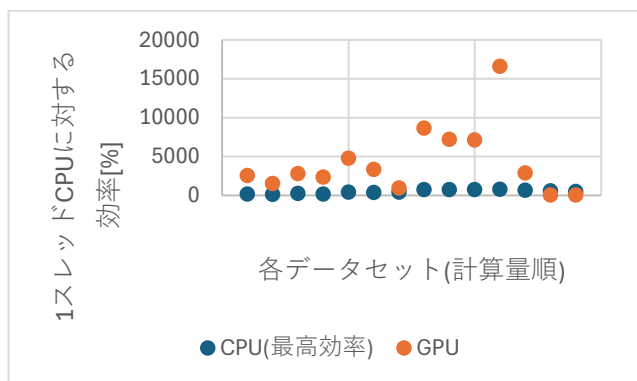


図3 GPU 並列実験とCPU 並列実験の比較

また、GPUのx-y次元を変化させ、計測を行った結果を図4に示す。

x-y	処理時間[秒]
32-32	154.731
64-16	226.927
128-8	183.814
256-4	241.606
512-2	295.466
1024-1	1513.72

図4 ブロックのx-y次元を変化させた場合の処理時間

4. むすび

図2より、CPUの並列実験においてスレッド数を1から順に上げた場合では処理が速くなることがわかる。しかしながら、データセットによってはスレッド数を上げすぎると、処理時間が遅くなるという結果も得られた。また、その現象は計算量が少ないデータセットに対して顕著に現れた。よって計算量が多いデータに対して最大限に並列化が可能となるが、スレッドが多すぎる場合では並列化による影響からオーバーヘッドによる影響を受けているとも考えられる。

次に、GPUの並列処理とCPUの並列処理の比較について、データ量が少ないデータセットに対してはCPUの方が高速で、データ量が多いデータセットに対してはGPUの方が高速になると予測されたものの、実験では図3に示したように、ほとんどの場合に対してGPUの方が高速となった。

また、図4よりブロックのx-y次元が32-32の場合が最も高速であるという結果となった。その理由は、MNISTの画像サイズは28x28であり、32x32のブロックであれば、画像データを完全に共有メモリにロードすることができる。これにより、グローバルメモリへのアクセスを完全に回避し、高速な共有メモリのみで処理を行うことが可能となる。

以上より、PNNのパターン分類に時間がかかるという問題に関して、本実験の各データセットに対するCPUの並列化によって高速化が可能であることがわかった。しかしながら、GPUでの処理が予想より時間がかかってしまったことよりCPUの方が優れているデータセットとGPUの方が優れているデータセットと区別することができなかった。その原因として、実験ではCPUおよびGPUそれぞれを用いた場合での並列処理プログラムを記述した言語が異なったため、内部処理が異なったことや、C++ではCPUとGPU両方の処理時間の計測を行ったが、その時点でCPUの方が高速だったことが多かったことなどが考えられる。よって、プログラムを同一の言語で実装し比較すること、およびプログラムの見直し、さらには、用いたPC環境の差異によるかどうかについての検証を行うことが今後の検討課題である。

参考文献

- 1) differences,”今さら聞けないGPUとCPUの違い”,<https://differences.jp/gpu-and-cpu/>
- 2) AI Market,”CUDAとは”,<https://ai-market.jp/technology/cuda/>
- 3) Specht D.F, “Probabilistic neural networks,” ELSEVIER, Neural Networks Vol.3,pp.109-118, (1990)
- 4) LAVIE”CPUのコアとスレッドって何?,”
https://www.nec-lavie.jp/products/contents/core_thread_pc.html
- 5) 清水伊織,”確率的ニューラルネットワークにおける並列化を用いた高速化について,”日本大学理工学部応用情報工学科卒業論文(2020)